

Homework 04: What A Method Can And Cannot Change

Two programs. The first makes you prove the pass-by-value rule to yourself instead of taking it on trust. The second walks you into a real production bug on purpose.

Part One: Inventory

Create `Inventory.java`. Start with a mutable list:

```
var stock = new ArrayList<String>(List.of("bolt", "nut"));
```

Write three methods and call all of them, printing the list after each:

1. `addItem(List<String> items, String item)` — adds an item. This one **must** be visible to the caller.
2. `replaceAll(List<String> items, List<String> replacements)` — written so that it assigns a new list to the parameter. This one **must not** be visible to the caller, and your printed output has to show that it is not.
3. `replaceAllProperly(...)` — the fixed version.

Then write a comment answering: there are exactly two ways to fix `replaceAll`. Name both, and say when you would pick each.

Part Two: The Remove Trap

Create `RemoveTrap.java`. Build a `List<Integer>` and trigger both overloads:

```
var ids = new ArrayList<Integer>(List.of(100, 200, 2, 300));  
int idToDrop = 2;  
ids.remove(idToDrop);
```

Print the result. It will not be what the code appears to say.

Then write a comment, **in your own words**, explaining why the compiler chose the overload it did. Your explanation has to mention where boxing sits in the resolution order, because that is the actual reason.

Finish by showing at least two ways to write it so it does what it looks like.

Acceptance Criteria

- ☐ Both files run with `java Inventory.java` and `java RemoveTrap.java`

- ☐ The printed output of `Inventory.java` proves `addItem` was visible to the caller and `replaceAll` was not
- ☐ A comment names both fixes for `replaceAll` and says when each applies
- ☐ `RemoveTrap.java` shows `remove(int)` and `remove(Object)` giving different results from the same starting list
- ☐ The explanation mentions overload resolution order, in your own words
- ☐ At least two correct alternatives are shown

Stretch

Add a small class holding a `List<String>` field with a getter that returns the field directly. From `main`, get the list, modify it, and print the object's contents to show you changed its internals from the outside without permission.

Then fix the getter, and say what the fix costs. There is more than one fix and they do not cost the same thing.

Hint, if part one looks like it already works

If `replaceAll` appears to change the caller's list, check whether you assigned to the parameter or called a method on it. Only one of those is visible from outside, and the whole exercise turns on which one you wrote.